# Mathematical practice final exam 2025

黄舟翔 3220103606

2025-07-02

```
## Warning: 程序包'tidyverse'是用R版本4.4.3 来建造的
## Warning: 程序包'ggplot2'是用R版本4.4.3 来建造的
## Warning: 程序包'tibble'是用R版本4.4.3 来建造的
## Warning: 程序包'tidyr'是用R版本4.4.3 来建造的
## Warning: 程序包'readr'是用R版本4.4.3 来建造的
## Warning: 程序包'purrr'是用R版本4.4.3 来建造的
## Warning: 程序包'dplyr'是用R版本4.4.3 来建造的
## Warning: 程序包'stringr'是用R版本4.4.3 来建造的
## Warning: 程序包'forcats'是用R版本4.4.3 来建造的
## Warning: 程序包'lubridate'是用R版本4.4.3 来建造的
## Warning: 程序包'Ecdat'是用R版本4.4.3 来建造的
## Warning: 程序包'Ecfun'是用R版本4.4.3 来建造的
```

## 1.

Find the inverse of the following matrix and verify it using the `all.equal()` function.

$$\begin{pmatrix} 9 & 4 & 12 & 2 \\ 5 & 0 & 7 & 9 \\ 2 & 6 & 8 & 0 \\ 9 & 2 & 9 & 11 \end{pmatrix}$$

用以下代码解决：

```r
# Define the original matrix
A <- matrix(c(9, 5, 2, 9,
              4, 0, 6, 2,
              12, 7, 8, 9,
              2, 9, 0, 11),
            nrow = 4, byrow = TRUE)

# Compute the inverse
A_inv <- solve(A)

# Print the inverse (rounded for clarity)
print("Inverse Matrix:")
```

```
## [1] "Inverse Matrix:"
```

```r
print(round(A_inv, 5))
```

```
##          [,1]     [,2]     [,3]     [,4]
## [1,]  0.08571 -0.14286  0.08571 -0.11429
## [2,] -0.26000 -0.30000  0.29000  0.03000
## [3,] -0.12286  0.17143  0.02714  0.04714
## [4,]  0.19714  0.27143 -0.25286  0.08714
```

```r
# Verification: Multiply A by A_inv
product <- A %*% A_inv

# Check if the product is the identity matrix
is_identity <- all.equal(product, diag(4))

# Print verification result
print("Verification Result:")
```

```
## [1] "Verification Result:"
```

```r
print(is_identity)
```

```
## [1] TRUE
```

## 2.

Execute the following lines which create two vectors of random integers which are chosen with replacement from the integers $0, 1, \ldots, 999$. Both vectors have length 250.

```
xVec <- sample(0:999, 250, replace=T)
yVec <- sample(0:999, 250, replace=T)
```

(a) Create the vector $(y_2 - x_1, \cdots, y_n - x_{n-1})$.

```
xVec <- sample(0:999, 250, replace = TRUE)
yVec <- sample(0:999, 250, replace = TRUE)
part_a <- yVec[-1] - xVec[-length(xVec)]
```

(b) Pick out the values in yVec which are $> 600$.

```
part_b <- yVec[yVec > 600]
print(part_b)
```

```
##   [1] 790 780 899 613 780 786 849 613 904 967 915 908 780 750 676 733 719 689
##  [19] 695 765 896 678 858 850 736 989 983 754 614 635 921 876 861 917 747 819
##  [37] 601 812 669 964 720 956 765 976 728 811 995 863 782 614 896 749 996 844
##  [55] 721 692 711 708 618 861 773 962 804 661 867 724 999 695 726 645 912 869
##  [73] 812 882 946 957 611 642 975 657 841 777 884 887 956 879 730 941 675 924
##  [91] 807 661 966 916 768 650 894 675 965 657
```

(c) What are the index positions in yVec of the values which are $> 600$?

```
part_c <- which(yVec > 600)
part_c
```

```
##   [1]   5   6   7   9  10  11  13  14  18  22  24  30  32  33  35  40  41  42
##  [19]  44  47  49  52  53  54  56  57  65  68  70  71  78  79  80  81  83  84
##  [37]  85  86  88  92  93  96  99 101 102 103 105 109 110 112 116 119 120 121
##  [55] 123 124 125 131 132 133 135 137 142 150 153 154 157 167 169 170 171 173
##  [73] 174 178 183 184 186 188 205 206 207 208 211 212 214 215 217 219 224 230
##  [91] 231 233 234 238 239 244 245 247 249 250
```

(d) Sort the numbers in the vector xVec in the order of increasing values in yVec.

```r
part_d <- xVec[order(yVec)]
```

(e) Pick out the elements in yVec at index positions $1, 4, 7, 10, 13, \cdots$

```r
indices <- seq(1, length(yVec), by = 3)
part_e <- yVec[indices]
part_e
```

```
##  [1] 113 222 899 780 849 376 360 967 166 518 403  79 209 733 277 195 896 678  57
## [20] 466 421 369 354 614 295  92 876 480 601 669 435 489  38 165 811 108 863 614
## [39] 335 362 844 692 402 391 861  43 410 804 123 510 547 724 999  12 244 442 726
## [58] 546 158 882 551 957 102 267 289 151  91 382 975 777 884 956 730 338 343 294
## [77] 521 206  53 916 387 650 675 657
```

### 3.

For this problem we'll use the (built-in) dataset state.x77.

```r
data(state)
state.x77 <- as_tibble(state.x77, rownames  = 'State')
```

a. Select all the states having an income less than 4300, and calculate the average income of these states.

```r
low_income_states <- state.x77 %>%
  filter(Income < 4300)

average_income_low <- mean(low_income_states$Income)
cat("a. 收入低于 4300 的州平均收入为:", round(average_income_low, 2), "\n\n")
```

```
## a. 收入低于4300的州平均收入为: 3830.6
```

b. Sort the data by income and select the state with the highest income.

```r
highest_income_state <- state.x77 %>%
  arrange(desc(Income)) %>%
  slice(1)
cat("b. 收入最高的州:\n")
```

```
## b. 收入最高的州:
```

```r
print(highest_income_state)
```

```
## # A tibble: 1 x 9
##   State  Population Income Illiteracy `Life Exp` Murder `HS Grad` Frost   Area
##   <chr>       <dbl>  <dbl>      <dbl>      <dbl>  <dbl>     <dbl> <dbl>  <dbl>
## 1 Alaska        365   6315        1.5       69.3   11.3      66.7   152 566432
```

```r
cat("\n")
```

c. Add a variable to the data frame which categorizes the size of population: $<= 4500$ is S, $> 4500$ is L.

d. Find out the average income and illiteracy of the two groups of states, distinguishing by whether the states are small or large.

```r
grouped_stats <- state_data %>%
  group_by(Pop_Size) %>%
  summarise(
    Avg_Income = mean(Income),
    Avg_Illiteracy = mean(Illiteracy)
  )
cat("d. 按人口规模分组的统计:\n")
```

```
## d. 按人口规模分组的统计:
```

```r
print(grouped_stats)
```

```
## # A tibble: 2 x 3
##   Pop_Size Avg_Income Avg_Illiteracy
##   <chr>         <dbl>          <dbl>
## 1 L             4608.           1.2
## 2 S             4355.           1.16
```

## 4.

a. Write a function to simulate n observations of $(X_1, X_2)$ which follow the uniform distribution over the square $[0, 1] \times [0, 1]$.

```r
simulate_points <- function(n) {
  # 生成 n 个在 [0,1]x[0,1] 正方形内的均匀分布点
  x1 <- runif(n, min = 0, max = 1)
  x2 <- runif(n, min = 0, max = 1)
  data.frame(x1 = x1, x2 = x2)
}
```

b. Write a function to calculate the proportion of the observations that the distance between $(X_1, X_2)$ and the nearest edge is less than 0.25, and the proportion of them with the distance to the nearest vertex less than 0.25.

```r
calculate_proportions <- function(points) {
  # 计算到最近边的距离
  dist_to_edges <- with(points, pmin(x1, 1 - x1, x2, 1 - x2))

  # 计算到四个顶点的距离
  dist_to_corners <- with(points, {
    d1 <- sqrt(x1^2 + x2^2)           # 到 (0,0) 的距离
    d2 <- sqrt(x1^2 + (1 - x2)^2)     # 到 (0,1) 的距离
    d3 <- sqrt((1 - x1)^2 + x2^2)     # 到 (1,0) 的距离
    d4 <- sqrt((1 - x1)^2 + (1 - x2)^2) # 到 (1,1) 的距离
    pmin(d1, d2, d3, d4)              # 取最小值得到最近顶点距离
  })

  # 计算比例
  prop_edge <- mean(dist_to_edges < 0.25)
  prop_vertex <- mean(dist_to_corners < 0.25)

  list(prop_edge = prop_edge, prop_vertex = prop_vertex)
}

# 测试函数
set.seed(123) # 确保结果可重现
n <- 10000    # 样本大小

# 模拟数据点
points_df <- simulate_points(n)

# 计算比例
```

```r
results <- calculate_proportions(points_df)

# 输出结果
cat(" 到最近边的距离 < 0.25 的比例:", round(results$prop_edge, 4), "\n")
```

```
## 到最近边的距离 < 0.25 的比例: 0.7493
```

```r
cat(" 到最近顶点的距离 < 0.25 的比例:", round(results$prop_vertex, 4), "\n")
```

```
## 到最近顶点的距离 < 0.25 的比例: 0.1948
```

### 5.

To estimate $\pi$ with a Monte Carlo simulation, we draw the unit circle inside the unit square, the ratio of the area of the circle to the area of the square will be $\pi/4$. Then shot $K$ arrows at the square, roughly $K * \pi/4$ should have fallen inside the circle. So if now you shoot $N$ arrows at the square, and $M$ fall inside the circle, you have the following relationship $M = N * \pi/4$. You can thus compute $\pi$ like so: $\pi = 4 * M/N$. The more arrows $N$ you throw at the square, the better approximation of $\pi$ you'll have.

```r
n <- 10000

set.seed(1)
points <- tibble("x" = runif(n), "y" = runif(n))
```

Now, to know if a point is inside the unit circle, we need to check whether $x^2 + y^2 < 1$. Let's add a new column to the points tibble, called `inside` equal to `1` if the point is inside the unit circle and `0` if not:

```r
points <- points |>
mutate(inside = map2_dbl(.x = x, .y = y, ~ifelse(.x**2 + .y**2 < 1, 1, 0))) |>
rowid_to_column("N")
```

a. Compute the estimation of $\pi$ at each row, by computing the cumulative sum of the 1's in the `inside` column and dividing that by the current value of `N` column:

```r
points <- points %>%
  mutate(
    cum_inside = cumsum(inside),  # 累积落入圆内的点数
    pi_estimate = 4 * cum_inside / N  #  的累积估计值
  )
tail(points, 5)
```

```
## # A tibble: 5 x 6
##       N     x     y inside cum_inside pi_estimate
##   <int> <dbl> <dbl>  <dbl>      <dbl>       <dbl>
## 1  9996 0.732 0.788      0       7824        3.13
## 2  9997 0.499 0.814      1       7825        3.13
## 3  9998 0.503 0.478      1       7826        3.13
## 4  9999 0.568 0.602      1       7827        3.13
## 5 10000 0.653 0.111      1       7828        3.13
```

b. Plot the estimates of $\pi$ against N.

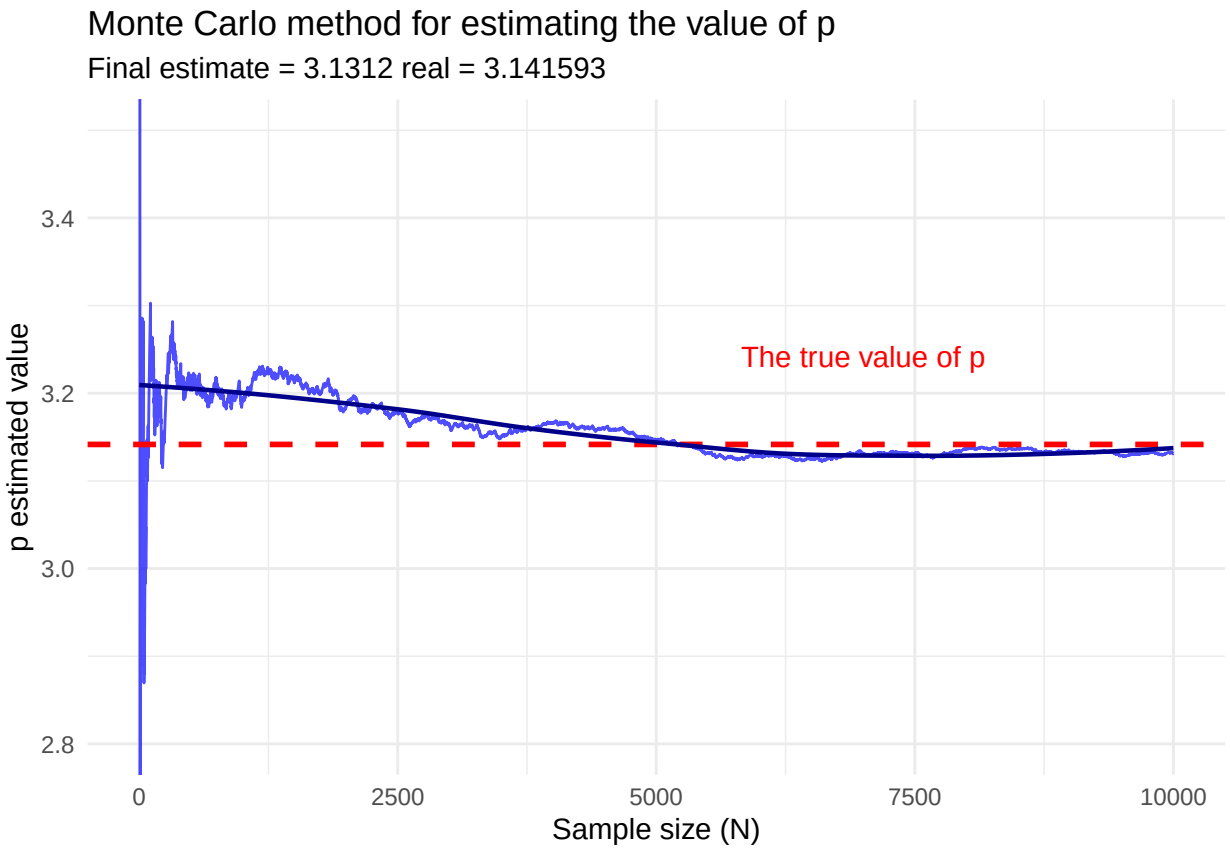
```
final_pi <- last(points$pi_estimate)
cat(" 最终  估计值:", round(final_pi, 6), "\n")
```

```
## 最终 估计值: 3.1312
```

```
# b. 绘制  估计值随 N 变化的曲线（不使用对数坐标）
ggplot(points, aes(x = N, y = pi_estimate)) +
  geom_line(color = "blue", alpha = 0.7) +
  geom_hline(yintercept = pi, color = "red", linetype = "dashed", size = 1) +
  labs(title = "Monte Carlo method for estimating the value of  ",
       subtitle = paste("Final estimate =", round(final_pi, 6), "real =", round(pi, 6)),
       x = "Sample size (N)",
       y = "  estimated value") +
  theme_minimal() +
  # 添加平滑曲线显示趋势
  geom_smooth(method = "loess", se = FALSE, color = "darkblue", size = 0.8) +
  # 添加说明文本
  annotate("text", x = n * 0.7, y = pi + 0.1,
           label = "The true value of  ", color = "red", size = 4) +
  # 限制 y 轴范围以更好地显示收敛
  coord_cartesian(ylim = c(2.8, 3.5))
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




**6.**

Mortality rates per 100,000 from male suicides for a number of age groups and a number of countries are given in the following data frame.

```
suicrates <- tibble(Country = c('Canada', 'Israel', 'Japan', 'Austria', 'France', 'Germany',
'Hungary', 'Italy', 'Netherlands', 'Poland', 'Spain', 'Sweden', 'Switzerland', 'UK', 'USA'),
Age25.34 = c(22,  9, 22, 29, 16, 28, 48,  7,  8, 26,  4, 28, 22, 10, 20),
Age35.44 = c(27, 19, 19, 40, 25, 35, 65,  8, 11, 29,  7, 41, 34, 13, 22),
Age45.54 = c(31, 10, 21, 52, 36, 41, 84, 11, 18, 36, 10, 46, 41, 15, 28),
Age55.64 = c(34, 14, 31, 53, 47, 49, 81, 18, 20, 32, 16, 51, 50, 17, 33),
Age65.74 = c(24, 27, 49, 69, 56, 52, 107, 27, 28, 28, 22, 35, 51, 22, 37))
```

a. Transform `suicrates` into *long* form.

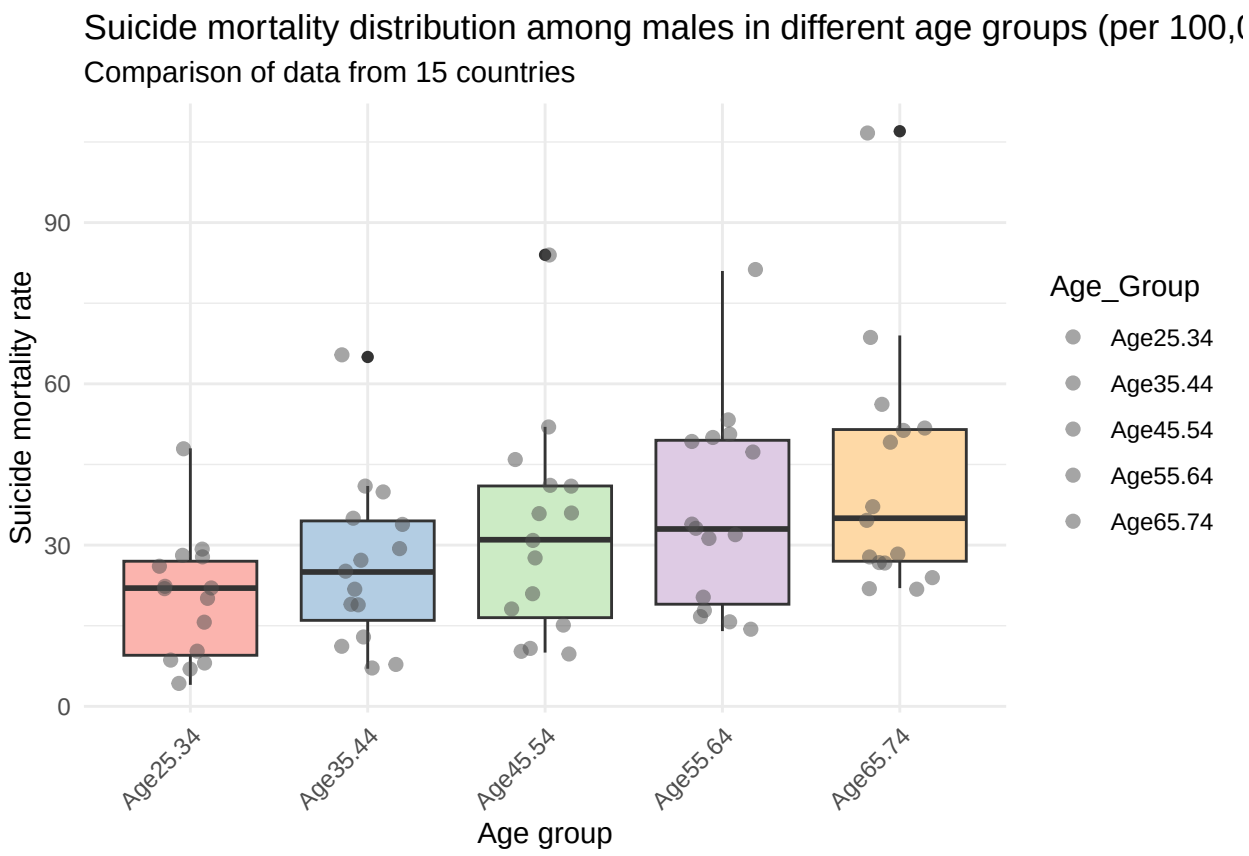
```
suicrates_long <- suicrates %>%
pivot_longer(
cols = starts_with("Age"),
names_to = "Age_Group",
```

```
values_to = "Mortality_Rate"
)
# 显示转换后的数据
head(suicrates_long)
```

```
## # A tibble: 6 x 3
##   Country Age_Group Mortality_Rate
##   <chr>   <chr>              <dbl>
## 1 Canada  Age25.34              22
## 2 Canada  Age35.44              27
## 3 Canada  Age45.54              31
## 4 Canada  Age55.64              34
## 5 Canada  Age65.74              24
## 6 Israel  Age25.34               9
```

b. Construct side-by-side box plots for the data from different age groups, and comment on what the graphic tells us about the data.

```
ggplot(suicrates_long, aes(x = Age_Group, y = Mortality_Rate, fill = Age_Group)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Suicide mortality distribution among males in different age groups (per 100,000 po
       subtitle = "Comparison of data from 15 countries",
       x = "Age group",
       y = "Suicide mortality rate") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Pastel1") +
  # 添加数据点显示实际分布
  geom_jitter(width = 0.2, alpha = 0.5, size = 2, color = "gray30")
```




```
# 分析数据
cat("\n数据分析:\n")
```

```
##
## 数据分析:
```

```
cat("1. 死亡率随年龄增长而上升：从 25-34 岁组到 55-64 岁组，死亡率中位数持续增加\n")
```

```
## 1. 死亡率随年龄增长而上升：从25-34岁组到55-64岁组，死亡率中位数持续增加
```

```
cat("2. 最高死亡率在 65-74 岁组：该组死亡率中位数最高，且分布范围最广\n")
```

```
## 2. 最高死亡率在65-74岁组：该组死亡率中位数最高，且分布范围最广
```

```
cat("3. 变异程度随年龄增加：老年组 (55-64 岁和 65-74 岁) 的数据范围更大，显示国家间差异更显著\n")
```

```
## 3. 变异程度随年龄增加：老年组(55-64岁和65-74岁)的数据范围更大，显示国家间差异更显著
```

```r
cat("4. 离群点：在 65-74 岁组观察到极高值 (>100)，可能是匈牙利的数据\n")
```

```
## 4. 离群点：在65-74岁组观察到极高值(>100)，可能是匈牙利的数据
```

```r
cat("5. 最低死亡率在年轻组：25-34 岁组整体死亡率最低，但国家间差异明显\n")
```

```
## 5. 最低死亡率在年轻组：25-34岁组整体死亡率最低，但国家间差异明显
```

```r
# 计算各年龄组统计量
age_group_stats <- suicrates_long %>%
  group_by(Age_Group) %>%
  summarise(
    Median = median(Mortality_Rate),
    Mean = mean(Mortality_Rate),
    SD = sd(Mortality_Rate),
    Min = min(Mortality_Rate),
    Max = max(Mortality_Rate)
  )

cat("\n各年龄组统计摘要:\n")
```

```
##
## 各年龄组统计摘要:
```

```r
print(age_group_stats)
```

```
## # A tibble: 5 x 6
##   Age_Group Median  Mean    SD   Min   Max
##   <chr>      <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Age25.34      22  19.9  11.5     4    48
## 2 Age35.44      25  26.3  15.3     7    65
## 3 Age45.54      31  32    19.9    10    84
## 4 Age55.64      33  36.4  18.7    14    81
## 5 Age65.74      35  42.3  23.0    22   107
```

**7.**

Load the `LaborSupply` dataset from the `{Ecdat}` package and answer the following questions:

```r
#data(LaborSupply)
LaborSupply <- read_csv("LaborSupply.csv")
```

```
## Rows: 5320 Columns: 7
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (7): lnhr, lnwg, kids, age, disab, id, year
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# create hour and wage variables
labor <- LaborSupply |>
  mutate(hour = exp(lnhr), wage = exp(lnwg), .before = kids) |>
  dplyr::select(-lnhr, -lnwg)
```

a. Compute the average annual hours worked and their standard deviations by year.

```r
hourly_stats <- labor %>%
  group_by(year) %>%
  summarise(
    avg_hours = mean(hour, na.rm = TRUE),
    sd_hours = sd(hour, na.rm = TRUE)
  )

cat("a. 按年份的平均工作小时数及标准差:\n")
```

```
## a. 按年份的平均工作小时数及标准差:
```

```r
print(hourly_stats)
```

```
## # A tibble: 10 x 3
##     year avg_hours sd_hours
##    <dbl>     <dbl>    <dbl>
##  1  1979     2202.     502.
##  2  1980     2182.     454.
##  3  1981     2185.     460.
##  4  1982     2145.     442.
##  5  1983     2124.     550.
```

```
##  6  1984     2149.     492.
##  7  1985     2203.     515.
##  8  1986     2195.     482.
##  9  1987     2219.     529.
## 10  1988     2222.     478.
```

b. What age group worked the most hours in the year 1982?

```
age_group_1982 <- labor |>
filter(year == 1982) |>
group_by(age) |>
  summarise(avg_hours = mean(hour)) |>
arrange(desc(avg_hours)) |>
slice(1)
cat(" 年龄:", age_group_1982$age, " 岁 | 平均工作时长:", round(age_group_1982$avg_hours, 1), " 小时
```

```
##  年龄: 46  岁 | 平均工作时长: 2373.5  小时
```

c. Create a variable, `n_years` that equals the number of years an individual stays in the panel. Is the panel balanced?

```
# 计算每个个体参与的年数
n_years <- labor %>%
  group_by(id) %>%
  summarise(n_years = n_distinct(year))

# 添加到主数据集
labor <- labor %>%
  left_join(n_years, by = "id")

# 检查面板是否平衡
is_balanced <- all(n_years$n_years == max(n_years$n_years))
cat("\nc. 面板是否平衡?", ifelse(is_balanced, " 是", " 否"), "\n")
```

```
##
## c. 面板是否平衡? 是
```

```
cat("   每个个体参与的年数范围:", range(n_years$n_years), "\n")
```

```
##    每个个体参与的年数范围: 10 10
```

d. Which are the individuals that do not have any kids during the whole period? Create a variable, `no_kids`, that flags these individuals (1 = no kids, 0 = kids)

```
# 找出整个期间都没有子女的个体
no_kids_ids <- labor %>%
  group_by(id) %>%
  summarise(always_no_kids = all(kids == 0)) %>%
  filter(always_no_kids) %>%
  pull(id)

# 创建 no_kids 变量
labor <- labor %>%
  mutate(no_kids = if_else(id %in% no_kids_ids, 1, 0))

cat("\nd. 整个期间无子女的个体数量:", length(no_kids_ids), "\n")
```

```
##
## d. 整个期间无子女的个体数量: 43
```

e. Using the `no_kids` variable from before compute the average wage, standard deviation and number of observations in each group for the year 1980 (no kids group vs kids group).

```
# e. 1980 年按有无子女分组的工资统计
wage_stats_1980 <- labor %>%
  filter(year == 1980) %>%
  group_by(no_kids) %>%
  summarise(
    avg_wage = mean(wage, na.rm = TRUE),
    sd_wage = sd(wage, na.rm = TRUE),
    n_obs = n()
  ) %>%
  mutate(no_kids = factor(no_kids, labels = c(" 有子女", " 无子女")))

cat("\ne. 1980 年按有无子女分组的工资统计:\n")
```

```
##
## e. 1980年按有无子女分组的工资统计:
```

```r
print(wage_stats_1980)
```

```
## # A tibble: 2 x 4
##   no_kids avg_wage sd_wage n_obs
##   <fct>      <dbl>   <dbl> <int>
## 1 有子女      14.5    6.69   489
## 2 无子女      15.9    6.71    43
```